

ECHO'S SIGNAL STUDIO — FULL CHEAT SHEET

Method in Motion · Events, Timing, Web Audio & Beat Machines

ECHO'S BIG SIGNAL IDEA

Something happens → code notices → code responds.

For music: choose what should play → move through time → trigger sound → repeat.

LESSON 1 — EVENTS & LISTENERS

EVENT

Analogy: pressing a doorbell or flipping a light switch.

Technical meaning: a moment the browser notices, such as click, keydown, input, mouse movement, or load.

EVENT-DRIVEN

Code that waits for events instead of running every response immediately.

EVENT LISTENER

The part of the code that waits for a chosen event.

addEventListener

Connects an element, an event type, and a callback.

Example:

```
btn.addEventListener("click", myFn);
```

CALLBACK FUNCTION

Analogy: a note that says, "When this happens, do this."

Technical meaning: instructions saved now so they can be called later.

EVENT HANDLER

The function that handles or responds to an event. Often the callback.

EVENT OBJECT

A small report the browser gives the callback with details about what happened.

Example: `e.clientX` can tell you the horizontal mouse position.

EVENT TYPE

The exact signal name being listened for.

Common examples: "click", "keydown", "input", "load".

READ THE LINE

```
btn.addEventListener("click", function() { ... });
```

`btn` = the element

`.` = reach into the element

`addEventListener` = wait for an event

`()` = hold the information being passed in

"click" = event type written as text

`,` = separates the event from the callback

`function() { ... }` = instructions to run later

`{ }` = function body

ECHO'S LESSON 1 RULE

Something happens → listener notices → callback responds.

LESSON 2 — TIMERS & TIMING

setTimeout

Analogy: a kitchen timer.

Waits for a delay, runs a callback once, then stops.

setInterval

Analogy: a repeating alarm clock.

Runs a callback again and again on a schedule until stopped.

clearInterval

Analogy: turning the repeating alarm off.

Stops the interval identified by its saved interval ID.

MILLISECOND (ms)

One thousandth of a second.

1000ms = 1 second.

DELAY

How long JavaScript waits before a scheduled action.

INTERVAL ID

The identifier returned by setInterval. Save it so clearInterval can stop the correct timer.

CALLBACK

The instructions a timer runs later or repeatedly.

BPM

Beats Per Minute.

BPM FORMULA

$60000 / \text{BPM} = \text{milliseconds per beat}$.

At 120 BPM:

$60000 / 120 = 500\text{ms per beat}$.

READ THE LINES

`setTimeout(function() { ... }, 3000);`

`setTimeout` = wait, then run once

`function() { ... }` = callback

3000 = 3-second delay

`const timer = setInterval(myFn, 500);`

`timer` = saved interval ID

`setInterval` = repeat

`myFn` = callback

500 = milliseconds between runs

`clearInterval(timer);`

`clearInterval` = stop repeating

`timer` = the saved interval to stop

ECHO'S LESSON 2 RULE

Once = `setTimeout`.

Repeat = `setInterval`.

Stop repeating = `clearInterval`.

LESSON 3 — THE WEB AUDIO API

WEB AUDIO API

The browser's built-in toolbox for creating and controlling audio with JavaScript.

AUDIOCONTEXT

Analogy: the whole sound studio or mixing board.

Technical meaning: the main browser audio environment where audio nodes are created and connected.

AUDIO NODE

One building block in an audio signal chain.

OSCILLATOR NODE

Analogy: one simple electronic instrument making one tone.

Technical meaning: an audio source that generates a waveform at a chosen frequency.

FREQUENCY

How fast a sound wave repeats; frequency controls pitch.

HERTZ (Hz)

The unit for frequency.

A4 = 440 Hz.

GAIN NODE

Analogy: a volume knob.

Technical meaning: a node that controls signal loudness.

Near 1 = louder; 0 = silent.

SIGNAL CHAIN

The path audio follows through connected nodes.

Source → processing → destination.

Example: oscillator → gain → speakers.

DESTINATION

The final audio output, usually speakers or headphones.

READ THE LINES

```
const ctx = new AudioContext();
```

ctx = short label for the studio

new = create a fresh instance

AudioContext() = create the browser audio studio

```
const osc = ctx.createOscillator();
```

```
osc.frequency.value = 440;
```

```
osc.connect(ctx.destination);
```

```
osc.start();
```

createOscillator() = create a tone source

frequency.value = 440 = set pitch to 440 Hz

connect(...) = route the signal

ctx.destination = final output

start() = begin the tone

```
const gain = ctx.createGain();
```

```
gain.gain.value = 0.4;
```

createGain() = create volume control

gain.gain.value = loudness number

0.4 = assigned volume

```
osc.connect(gain);
```

```
gain.connect(ctx.destination);
```

First: tone → volume control.

Then: volume control → output.

ECHO'S LESSON 3 RULE

Create → set values → connect → start.

LESSON 4 — BEAT MACHINE

STEP SEQUENCER

Analogy: a music box with pins on a spinning cylinder.

Technical meaning: a system that divides time into equal steps and triggers sounds on active steps.

GRID DATA

The stored rows and steps that remember which sounds are active.

BOOLEAN

A value that can only be true or false.

In the grid: true = on, false = off.

TOGGLE

Flip between two states: false $\leftrightarrow$ true.

STEP

One position in the sequencer timeline.

setInterval IN THE SEQUENCER

Acts like the motor, moving the sequencer forward one step at a time.

BPM IN THE SEQUENCER

Controls how quickly the steps move. Cross-reference: Lesson 2.

STEP WRAP

Returning to step 0 after the final step so the pattern repeats.

REMAINDER OPERATOR (%)

Returns the remainder after division and is useful for wrapping counters.

STEP-WRAP FORMULA

`step = (step + 1) % 16;`

step = save the next step

(step + 1) = move forward one

% 16 = keep the value cycling through 0–15

After 15, the next value wraps to 0.

TOGGLING GRID DATA

`beatGrid[row][step] = !beatGrid[row][step];`

beatGrid = stored grid

[row][step] = one exact square

= = assign a new value

! = logical NOT; flips true to false or false to true

THE FULL MACHINE

Click event → boolean changes → timer reaches step → active sound triggers → Web Audio creates sound.

ECHO'S LESSON 4 RULE

Choose the pins → move one step at a time → play what is active → wrap and repeat.

SIGNAL CHAIN SYMBOLS

()

Parentheses. Call a function, hold arguments, or group math.

.

Dot. Access a property or method.

Example: `ctx.createOscillator()`

" " or ' '

Quotes. Wrap text values.

Examples: "click", 'keydown'

=

Assignment. Store or replace a value.

,

Comma. Separate arguments or items.

{ }

Braces. Wrap a function body or code block.

%

Remainder operator. Used in the step-wrap formula.

[]

Brackets. Access array or grid positions.

Example: beatGrid[row][step]

!

Logical NOT. Flips a boolean when used as !value.

//

Starts a single-line comment.

+

Adds numbers or joins text values depending on the data.

/

Division operator.

Example: 60000 / BPM

QUICK RECALL

Doorbell → event analogy

Listener → waits

Callback → instructions for later

setTimeout → once

setInterval → repeat

clearInterval → stop repeat

1000ms → 1 second

60000 / BPM → milliseconds per beat

AudioContext → whole studio

OscillatorNode → tone-making instrument

GainNode → volume knob

440 Hz → A4

Signal chain → source → processing → destination

Step sequencer → digital music box

Boolean → true or false

% 16 → wraps a 16-step counter

!value → flips a boolean

ECHO'S FINAL SIGNAL

EVENTS tell code **WHEN**.

TIMERS control **WHEN** AGAIN.

WEB AUDIO controls **WHAT YOU HEAR**.

THE SEQUENCER combines all three into a repeating musical system.